

Transfer Learning

July 10, 2019

```
In [9]: import torch
import torch.nn as nn
import torch.nn.functional as f
from torch.autograd import Variable as V

from torch.utils.data import Dataset, DataLoader

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

import os
from torchvision.transforms import transforms
from torch.utils.data import DataLoader
```

```
In [2]: import torchvision.models as models
```

0.1 Methods

```
In [3]: def fit(model, epoch):
    for j in range(epoch):
        train_loss = 0
        val_loss = 0
        for i, data in enumerate(trn_dl):
            opt.zero_grad()

            data = data.type(torch.float).cuda()
            out = model(V(data))

            loss = f.binary_cross_entropy(out, data)
            loss.backward()
            opt.step()

            train_loss += loss.data

        for i, data in enumerate(val_dl):
```

```

        data = data.type(torch.float).cuda()
        out = model(V(data))
        loss = f.binary_cross_entropy(out,data)
        val_loss += loss

    model.record(loss)
    train_loss /= len(trn_dl)
    val_loss /= len(val_dl)
    print('train Loss: ', train_loss.tolist(), ' validation loss: ',val_loss.tolist())

```

0.1.1 Get Dataset

```

In [4]: dataset = np.load('dataset_s112.npy')

In [5]: dataset = np.array([i.reshape(1,112,112) for i in dataset])

In [6]: dataset.shape

Out[6]: (10000, 1, 112, 112)

```

0.1.2 Split Data Train/Val

```

In [7]: def split(dataset,prc=0.2):
        size = len(dataset)
        size = int(size*prc)

        val = dataset[-size:]
        trn = dataset[:-size]
        return trn , val

In [8]: trn , val = split(dataset)

```

0.1.3 Prepare DataLoaders

```

In [9]: trn_dl = DataLoader(trn[:,],batch_size=32,num_workers=2,shuffle=False)
        val_dl = DataLoader(val[:,1000],batch_size=32,num_workers=5,shuffle=False)

```

0.2 Model Component

```

In [10]: class res(nn.Module):
        def __init__(self):
            super().__init__()
            self.loss = []
            self.pre = models.resnet18(pretrained=True)
            self.deconv1 = nn.ConvTranspose2d(1,1,(11,16))
            self.deconv2 = nn.ConvTranspose2d(1,1,(21,31))
            self.deconv3 = nn.ConvTranspose2d(1,1,(21,21))
            self.deconv4 = nn.ConvTranspose2d(1,1,(23,23))

```

```

def forward(self,inputs):
    bs = inputs.size(0)
    #print(bs)
    inputs = torch.cat([inputs,inputs,inputs],dim=1)
    k = self.pre(inputs)
    k = k.view(bs,40,-1)
    k = k[:,None,:,:]
    k = f.elu(self.deconv1(k))
    k = f.elu(self.deconv2(k))
    k = f.elu(self.deconv3(k))

    return torch.sigmoid(self.deconv4(k))
def record(self,record):
    self.loss.append(record)

```

```

In [11]: m = res().cuda()
         opt = torch.optim.Adam(m.parameters(),1e-2)

```

```

In [12]: # Freezing
         for i in list(m.parameters())[:-8]:
             i.requires_grad_(False)

```

```

In [13]: fit(m,9)

```

train Loss:	0.17308036983013153	validation loss:	0.09877319633960724
train Loss:	0.09535352885723114	validation loss:	0.09246526658535004
train Loss:	0.09078678488731384	validation loss:	0.08894288539886475
train Loss:	0.08777286112308502	validation loss:	0.0863366350531578
train Loss:	0.08541208505630493	validation loss:	0.0842040404677391
train Loss:	0.08351971954107285	validation loss:	0.08244214206933975
train Loss:	0.0820433646440506	validation loss:	0.08115046471357346
train Loss:	0.08094283193349838	validation loss:	0.08017091453075409
train Loss:	0.08013035356998444	validation loss:	0.07955366373062134

```

In [14]: # UnFreezing
         for i in list(m.parameters())[:-8]:
             i.requires_grad_(True)

```

```

In [15]: fit(m,3)

```

train Loss:	0.07863686978816986	validation loss:	0.07735799252986908
train Loss:	0.07661574333906174	validation loss:	0.07530385255813599
train Loss:	0.07445866614580154	validation loss:	0.07288000732660294

```

In [16]: fit(m,9)

```

```

train Loss: 0.07181785255670547    validation loss: 0.07045558094978333
train Loss: 0.06974868476390839    validation loss: 0.06872431933879852
train Loss: 0.06789860129356384    validation loss: 0.06685392558574677
train Loss: 0.06636039167642593    validation loss: 0.06543124467134476
train Loss: 0.06501849740743637    validation loss: 0.06422445923089981
train Loss: 0.06376167386770248    validation loss: 0.06303326040506363
train Loss: 0.06261036545038223    validation loss: 0.06254146993160248
train Loss: 0.06158936023712158    validation loss: 0.06159556657075882
train Loss: 0.06045856326818466    validation loss: 0.060137953609228134

```

```
In [17]: #Increase Learning rate something like SGDR
```

```

for i in opt.param_groups:
    i['lr'] = 1e-1/2

```

```
In [18]: fit(m,2)
```

```

train Loss: 0.34798377752304077    validation loss: 0.16904619336128235
train Loss: 0.3808107376098633    validation loss: 0.3375665545463562

```

```
In [19]: #Decrease Learning rate
```

```

for i in opt.param_groups:
    i['lr'] = 1e-2

```

```
In [20]: fit(m,6)
```

```

train Loss: 0.34194323420524597    validation loss: 0.3368692398071289
train Loss: 0.33024418354034424    validation loss: 0.31694573163986206
train Loss: 0.315679669380188      validation loss: 0.3057340085506439
train Loss: 0.3012155592441559      validation loss: 0.29640674591064453

```

RuntimeError

Traceback (most recent call last)

<ipython-input-20-1cda4d5c0ead> in <module>

----> 1 fit(m,6)

<ipython-input-3-247b49bbdc69> in fit(model, epoch)

18

19 data = data.type(torch.float).cuda()

----> 20 out = model(V(data))

21 loss = f.binary_cross_entropy(out,data)

22 val_loss += loss

```

~/anaconda3/envs/fastv1/lib/python3.7/site-packages/torch/nn/modules/module.py in __call__
487         result = self._slow_forward(*input, **kwargs)
488     else:
--> 489         result = self.forward(*input, **kwargs)
490     for hook in self._forward_hooks.values():
491         hook_result = hook(self, input, result)

<ipython-input-10-083000f84552> in forward(self, inputs)
13         #print(bs)
14         inputs = torch.cat([inputs,inputs,inputs],dim=1)
---> 15         k = self.pre(inputs)
16         k = k.view(bs,40,-1)
17         k = k[:,None,:,:]

~/anaconda3/envs/fastv1/lib/python3.7/site-packages/torch/nn/modules/module.py in __call__
487         result = self._slow_forward(*input, **kwargs)
488     else:
--> 489         result = self.forward(*input, **kwargs)
490     for hook in self._forward_hooks.values():
491         hook_result = hook(self, input, result)

~/anaconda3/envs/fastv1/lib/python3.7/site-packages/torchvision/models/resnet.py in forward
149     def forward(self, x):
150         x = self.conv1(x)
--> 151         x = self.bn1(x)
152         x = self.relu(x)
153         x = self.maxpool(x)

~/anaconda3/envs/fastv1/lib/python3.7/site-packages/torch/nn/modules/module.py in __call__
487         result = self._slow_forward(*input, **kwargs)
488     else:
--> 489         result = self.forward(*input, **kwargs)
490     for hook in self._forward_hooks.values():
491         hook_result = hook(self, input, result)

~/anaconda3/envs/fastv1/lib/python3.7/site-packages/torch/nn/modules/batchnorm.py in forward
74         input, self.running_mean, self.running_var, self.weight, self.bias,
75         self.training or not self.track_running_stats,
---> 76         exponential_average_factor, self.eps)
77
78     def extra_repr(self):

```

```

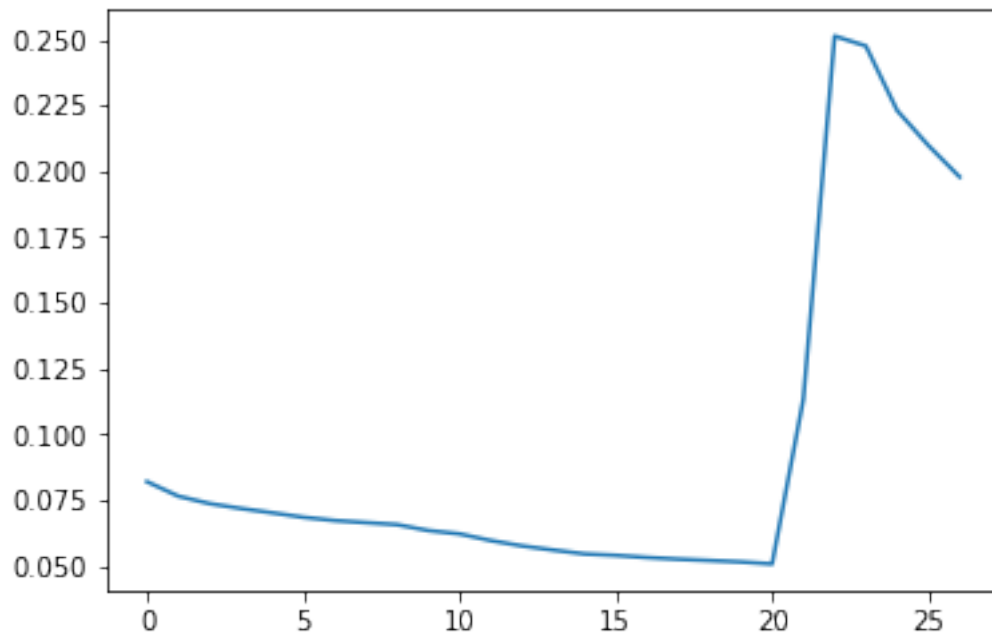
~/anaconda3/envs/fastv1/lib/python3.7/site-packages/torch/nn/functional.py in batch_norm
1621     return torch.batch_norm(
1622         input, weight, bias, running_mean, running_var,
-> 1623         training, momentum, eps, torch.backends.cudnn.enabled
1624     )
1625

```

RuntimeError: CUDA out of memory. Tried to allocate 6.12 MiB (GPU 0; 7.93 GiB total cap

```
In [21]: plt.plot(m.loss)
```

```
Out[21]: [<matplotlib.lines.Line2D at 0x7f02d96afd68>]
```



```
In [22]: np.save('records/resnet_32_3.npy', m.loss)
```

0.3 Conclusion

```
In [10]: a = os.listdir('records/')

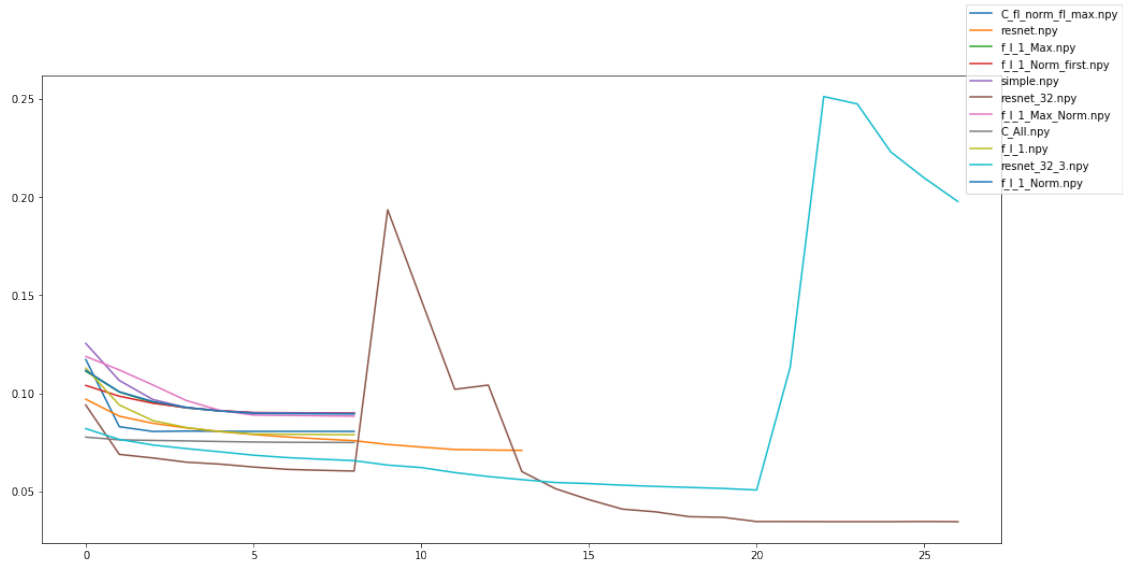
```

```

In [11]: fig, ax = plt.subplots(figsize=(16,8))
        for i in a:
            b = np.load('records/'+i)
            ax.plot(b, label=i)
        fig.legend()

```

Out[11]: <matplotlib.legend.Legend at 0x7f49d26a66a0>



Transfer Learning models consist of pretrained model as an encoder + 4 upsampling layers as decoder In This image we compare between simple models developed in the previous notebook and the pretrained model The Pre-trained model looks very promising but needs more time and computation power The brown line peak represent the loss when the model was fully unfreezed The blue line peak represent the loss when the learning rate was increased this is influenced from SGDR but the model looks that it totally lost its way.

IMP NOTE: the pre-trained model is trained on even smaller part of the validation set data (not the same to previous models) due to cuda errors.Smaller batch sizes didn't solve the errors